

摘要：

DeepSeek悄然更新V4版本，推出投机解码框架DSpark并开源全栈工具DeepSpec。
DSpark将并行生成与自适应负载感知验证结合，在保持相同吞吐量前提下，用户生成速度
最高提升85%，平均接受长度全面超越Eagle3和DFlash。

刚刚，DeepSeek V4 进行了一次更新。

新推出了投机解码（Speculative Decoding）框架 DSpark，并同步开源了支撑该版本的全栈推测性解码框架 DeepSpec。

DeepSeek-V4-Pro-DSpark 并非全新架构模型，而是在 DeepSeek-V4-Pro 基础上引入了推测性解码模块。此次更新的重点在于工程落地，而非模型能力本身的迭代。

DSpark 已被部署在 DeepSeek-V4（Flash 和 Pro）的真实线上流量中，大幅加速了大语言模型（LLM）的推理速度。



DSpark: Confidence-Scheduled Speculative Decoding with Semi-Autoregressive Generation

Xin Cheng^{1,2*}, Xingkai Yu^{2*}, Chenze Shao^{2*}, Jiashi Li^{2*}, Yunfan Xiong^{2*},
Yi Qian², Jiaqi Zhu², Shirong Ma², Xiaokang Zhang², Jiasheng Ye², Qinyu Chen²,
Chengqi Deng², Jiping Yu², Damai Dai², Zhengyan Zhang², Yixuan Wei², Yixuan Tan²,
Wenkai Yang², Runxin Xu², Yu Wu², Zhean Xu², Xuanyu Wang², Muyang Chen²,
Rui Tian², Xiao Bi², Zhewen Hao², Shaoyuan Chen², Huanqi Cao², Wentao Zhang²,
Anyi Xu², Huishuai Zhang¹, Dongyan Zhao¹, Wenfeng Liang²
¹Peking University ²DeepSeek-AI
{chengxin, xing kai, shaochenze, js.li, yunfanxiong}@deepseek.com

Abstract

Speculative decoding accelerates Large Language Model (LLM) inference by decoupling draft generation from target verification. While recent parallel drafters efficiently propose long token sequences in a single forward pass, they suffer from rapid acceptance decay due to a lack of inter-token dependencies. Furthermore, indiscriminately verifying these extended blocks wastes critical batch capacity on tokens with high rejection risks, severely degrading throughput in high-concurrency serving systems. We introduce DSpark, a speculative decoding framework that unifies high-throughput parallel generation with adaptive, load-aware verification. To maintain draft quality, DSpark utilizes a semi-autoregressive architecture—coupling a parallel backbone with a lightweight sequential module—to introduce intra-block dependency modeling and mitigate suffix decay. To optimize system efficiency, DSpark employs confidence-scheduled verification, dynamically tailoring the verification length for each request based on estimated prefix survival probabilities and engine-specific throughput profiles. On offline benchmarks across diverse domains, DSpark substantially improves the accepted length over state-of-the-art autoregressive and parallel drafters. When deployed within the DeepSeek-V4 serving system under live user traffic, DSpark successfully mitigates verification waste. Compared to the established production baseline (MTP-1), DSpark accelerates per-user generation speeds by 60%–85% at matched throughput levels. More importantly, by preventing severe throughput degradation under strict interactivity constraints, it enables performance tiers that were previously unattainable, shifting the Pareto frontier of our serving system. To facilitate community progress, we open-source the [DSpark checkpoints](#) alongside [DeepSpec](#), an algorithm-driven training repository for speculative decoding.

- 技术报告：《DSpark: Confidence-Scheduled Speculative Decoding with Semi-Autoregressive Generation》
- 技术报告链接：https://github.com/deepseek-ai/DeepSpec/blob/main/DSpark_paper.pdf

DSpark 的核心初衷是解决在生产环境中（尤其是高并发场景下）LLM 推理面临的延迟和吞吐量瓶颈。简而言之，DSpark 成功地将高吞吐量的「并行生成」与自适应的「负载感知验证」结合在了一起。

推测性解码是一种在不改变模型输出分布的前提下加速大语言模型推理的技术。其核心思路是引入一个轻量级的「草稿模型」（draft model），预先生成若干候选 token，再由目标模型（target model）对这批候选进行批量验证和接受，从而将串行逐 token 生成转变为并行批量校验，大幅降低端到端延迟。

在此基础上，DSpark 的创新在于引入半自回归生成架构（Semi-Autoregressive Generation）：它保留并行草稿模型的高吞吐优势，同时加入轻量级串行模块，对 block 内 token 之间的依赖关系进行建模，以缓解并行草稿模型在后续位置上容易出现的接受率衰减问题。

除此之外还有硬件感知的置信度调度验证（Confidence-Scheduled Verification）：以往的投机解码通常会盲目地把生成的草稿 Token 全部送去验证，在系统高负载时，这些极大概率会被拒绝的尾部 Token 会严重浪费宝贵的批处理算力。DSpark 引入了一个置信度头（Confidence Head）来评估每个 Token 的存活概率。结合硬件感知前缀调度器，系统能够根据实时的引擎吞吐量特征，动态为每个请求量身定制最优的验证长度，将算力只分配给预期回报最高的 Token。

为了在真实的线上基础设施中落地，DSpark 的调度器采用了异步机制，以兼容零开销调度（ZOS）和连续的 CUDA 图回放。它利用前两步的历史预测来决定当前的动态截断长度，从而隐藏了调度延迟，避免了 GPU 流水线停顿，同时保证了目标模型输出分布的完全无损还原。

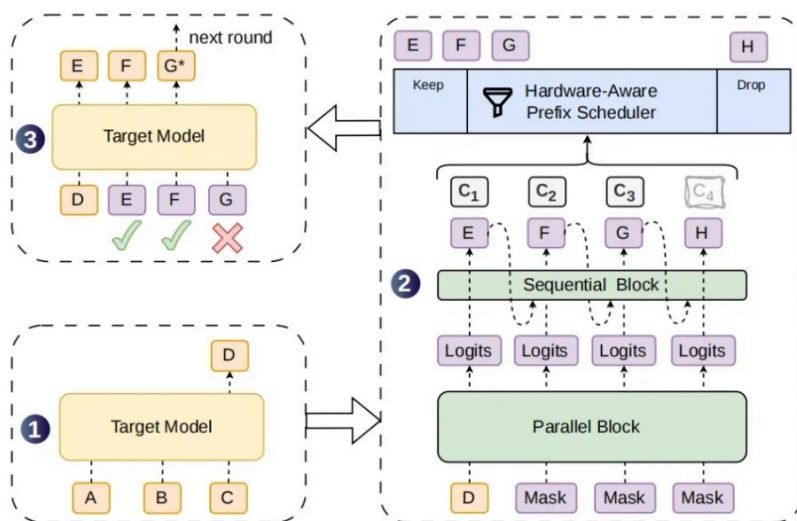


Figure 1 | The DSpark architecture and decoding cycle. Given prompt tokens `ABC`, the target model executes one step to generate the next token `D`, which serves as the anchor for the drafting phase. Using `D` as the input, DSpark employs a heavy parallel backbone and a lightweight sequential head to generate draft tokens `EFGH` along with their corresponding confidence scores `c1-c4`. The Hardware-Aware Prefix Scheduler then evaluates these scores to retain the prefix `EFG` and drop the low-confidence token `H`. Finally, the target model verifies the scheduled prefix in parallel. As illustrated, `E` and `F` are accepted while `G` is rejected, prompting the model to generate a corrected token `G*` to complete the current round.

在涵盖数学推理、代码生成和日常对话等多个领域的测试中，DSpark 大幅超越了目前最先进的自回归模型（Eagle3）和并行草稿模型（DFlash）。例如，在 Qwen3 系列（4B、8B、14B）目标模型上，其平均接受长度比 Eagle3 提升了 26.7% 到 30.9%，比 DFlash 提升了 16.3% 到 18.4%。

Table 1 | **Main speculative decoding results.** We report accepted length (τ) per decoding round (higher is better) for different target models and domains. **Bold** marks the best results.

Target	Drafter	Math			Code			Chat		
		GSM8K	MATH	AIME25	MBPP	HumanEval	LCB	MT-Bench	Alpaca	Arena-Hard
Qwen3-4B	Eagle3	5.14	4.62	3.92	3.69	4.16	3.77	2.39	2.26	2.55
	DFlash	5.40	4.85	4.15	4.40	4.74	4.18	3.07	2.96	2.83
	DSpark	6.11	5.70	4.89	5.13	5.38	4.86	3.64	3.54	3.29
Qwen3-8B	Eagle3	5.30	4.77	3.91	3.96	4.33	4.17	2.66	2.54	2.54
	DFlash	5.33	4.91	4.07	4.36	4.64	4.39	3.11	2.98	2.81
	DSpark	6.17	5.78	5.01	5.16	5.52	5.17	3.72	3.58	3.21
Qwen3-14B	Eagle3	5.24	4.60	3.71	3.81	4.14	4.01	2.62	2.47	2.48
	DFlash	5.41	4.84	3.98	4.44	4.59	4.33	3.10	2.94	2.72
	DSpark	6.21	5.74	4.94	5.26	5.43	5.02	3.70	3.58	3.13
Gemma4-12B	Eagle3	5.87	5.46	4.83	4.72	5.37	4.16	3.19	3.06	2.72
	DFlash	5.45	5.04	4.22	4.39	4.95	3.70	2.98	2.84	2.59
	DSpark	6.05	5.78	5.12	5.11	5.64	4.51	3.49	3.35	2.92

相比于前一代部署的单 Token 生产基准（MTP-1），在维持相同总体吞吐量的情况下，DSpark 将用户的生成速度分别提升了 60%-85%（Flash 模型）和 57%-78%（Pro 模型）。

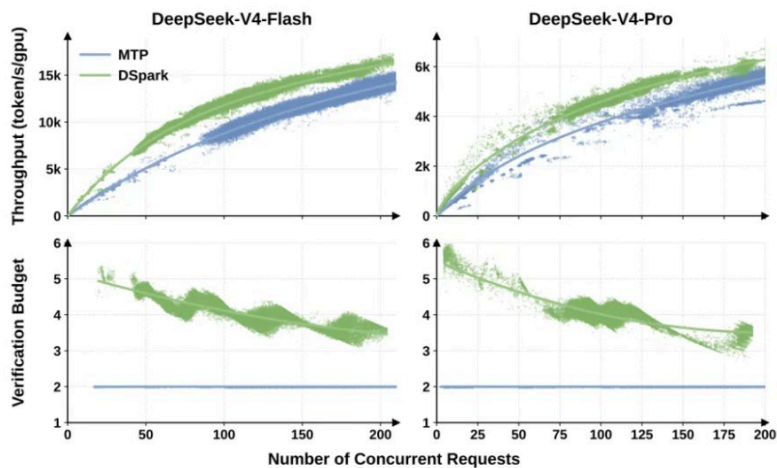


Figure 8 | **Load-adaptive throughput and verification budgets.** Top row (a, b): Aggregate output throughput across varying levels of system concurrency. Bottom row (c, d): The average target verification budget allocated per request. As concurrent load increases, the dynamic scheduler automatically restricts the per-request verification length to prevent resource contention.

随 DSpark 一同开源的还有 DeepSpec，这是一个用于训练和评估推测性解码草稿模型的全栈代码库。是承载这个方案以及其他前沿算法实现的「开源基础设施」，包含数据准备工具、草稿模型实现、训练代码和评估脚本。

DeepSpec 将整体流程拆分为三个阶段：数据准备、训练和评估。三个阶段需要按顺序运行，前一阶段的输出会作为后一阶段的输入。

数据准备阶段，需下载提示词数据、使用推理引擎对目标模型重新生成答案，并构建目标缓存（target cache）。值得注意的是，以默认的 Qwen/Qwen3-4B 配置为例，目标缓存体积可达约 38 TB，使用前需充分评估存储资源。

训练阶段可通过 `bash scripts/train/train.sh` 启动。该脚本会调用 `train.py`，并为每张可见 GPU 启动一个 worker。用户可以通过指定 `config_path`，在 `config/` 目录下选择不同算法和目标模型配置。项目也支持通过覆盖 `config_path`、`target_cache_dir`，以及使用 `--opts` 修改单个配置字段来调整训练设置。

硬件方面，DeepSpec 默认配置和脚本面向单节点 8 卡环境。如果 GPU 数量较少，用户需要相应减少 `CUDA_VISIBLE_DEVICES` 中的可见 GPU 数量。

评估阶段则通过 `bash scripts/eval/eval.sh` 启动。评估脚本会使用训练好的草稿模型 checkpoint，在多个 speculative decoding 基准任务上衡量接受情况。项目当前列出的评估数据



集包括 GSM8K、MATH500、AIME25、HumanEval、MBPP、LiveCodeBench、MT-Bench、Alpaca 和 Arena-Hard-v2，覆盖数学推理、代码生成、对话能力和综合问答等不同任务类型。

算法方面，DeepSpec 目前内置三种草稿模型：DSpark、DFlash 和 Eagle3。目标模型系列方面，项目当前支持 Qwen3 和 Gemma。

DeepSpec 的开源，将推测性解码这一此前多散落于各研究团队内部的工程实践，整合为一套可复现、可扩展的标化工具链。对于希望为自有大模型加速推理的研究者和工程师而言，这意味着可以直接在成熟框架上训练定制草稿模型，跳过大量重复的基础设施搭建工作。

本文来源：[机器之心](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

* 体验资格每人限申领一次

扫码

免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

803 收藏

分享到:

写评论

请发表您的评论

表情 图片

发表评论